OBS STUDIO
QT DOCK PLUGIN DEVELOPMENT

Volume II | Native C++17 + Qt Widgets integration

Project: OBS Orientation Switcher

Environment: OBS Studio 32.2.1 | Qt 6.11.1 | Visual Studio 2022 | Windows x64

Status: Release x64 DLL compiled; IntelliSense rescanned successfully. Dock runtime testing remains outstanding.

A practical companion guide documenting the development environment, MSBuild property sheet, warning isolation, IntelliSense recovery, dock architecture, deployment, validation, and troubleshooting.

Edition: 6 November 2026

1. Purpose and confirmed milestones

This second volume documents the Qt-enabled variant of the OBS Orientation Switcher plugin. The earlier minimal native plugin used OBS frontend APIs and Tools-menu commands. This variant adds a Qt Widgets user interface intended to be hosted as a dock within OBS Studio.

• Existing native plugin and Tools-menu switching were tested successfully before Qt integration.

• Qt 6.11.1 runtime DLLs were identified in the installed OBS bin\64bit directory.

• The Qt development SDK was acquired separately and its include and library directories integrated into the project.

• After adding C++17 and /Zc:__cplusplus, Release x64 rebuilt successfully and produced the DLL.

• Visual Studio IntelliSense errors cleared after rescanning the solution.

• Successful compilation does not itself establish successful dock registration, correct UI-thread ownership, or safe profile switching at runtime.

2. Directory layout and toolchain

```
E:\VSCODE\obs-orientation-switcher\
  OBSPaths.props
  lib\                 # Minimal OBS headers and import libs
  qt\
    include\           # QtCore, QtGui, QtWidgets, etc.
    lib\               # Qt6Core.lib, Qt6Gui.lib, Qt6Widgets.lib
    mkspecs\           # Qt build specifications (retained)
  src\plugin-main.cpp
  build\x64\Release\obs-orientation-switcher.dll
```

OBS source tree: E:\obs-studio-32.2.1. Installed OBS: C:\Program Files\obs-studio. The installed Qt DLLs are runtime binaries; development headers and import libraries were not found in the original scanned directories.

Required Qt components

Module	Headers	Link library
QtCore	QTimer, QObject	Qt6Core.lib
QtGui	GUI framework dependencies	Qt6Gui.lib
QtWidgets	QWidget, QPushButton, QLabel, layouts	Qt6Widgets.lib

The Qt package should be built for the compatible MSVC x64 ABI and should match the installed runtime version (6.11.1) as closely as possible. Do not replace the Qt runtime DLLs bundled with OBS without a separate compatibility assessment.

3. Recommended MSBuild property sheet

The following preserves standard include directories for the compiler and IntelliSense and uses MSVC external-header warning treatment for angle-bracket includes. This is the configuration associated with the successful build and IntelliSense rescan.

```
<?xml version="1.0" encoding="utf-8"?>
<Project xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
  <PropertyGroup>
```

```
    <OBSSDKDir>$(MSBuildThisFileDirectory)lib\</OBSSDKDir>
    <QtSDKDir>$(MSBuildThisFileDirectory)qt\</QtSDKDir>
  </PropertyGroup>
  <ItemDefinitionGroup>
    <ClCompile>
      <LanguageStandard>stdcpp17</LanguageStandard>
      <ConformanceMode>true</ConformanceMode>
      <WarningLevel>Level4</WarningLevel>
      <AdditionalIncludeDirectories>$(OBSSDKDir);$(QtSDKDir)include;
$(QtSDKDir)include\QtCore;$(QtSDKDir)include\QtGui;
$(QtSDKDir)include\QtWidgets;%(AdditionalIncludeDirectories)</AdditionalIncludeDirectories>
      <AdditionalOptions>/Zc:__cplusplus /external:anglebrackets
/external:W0 %(AdditionalOptions)</AdditionalOptions>
    </ClCompile>
    <Link>
      <AdditionalLibraryDirectories>$(OBSSDKDir);$(QtSDKDir)lib;
%(AdditionalLibraryDirectories)</AdditionalLibraryDirectories>
      <AdditionalDependencies>obs.lib;obs-frontend-api.lib;
Qt6Core.lib;Qt6Gui.lib;Qt6Widgets.lib;
%(AdditionalDependencies)</AdditionalDependencies>
    </Link>
  </ItemDefinitionGroup>
</Project>
```

XML note: The wrapped lines in the printed listing are for readability. In the actual property sheet, keep each semicolon-separated MSBuild property value on a single line to avoid unintended whitespace or line-break effects.

4. Compiler diagnostics and warning isolation

Language settings

Qt 6.11.1 requires C++17 and a correct MSVC __cplusplus value. Set LanguageStandard=stdcpp17 and add /Zc:__cplusplus. The original compiler failure occurred in qcompilerdetection.h with C1189; adding these settings resolved that failure.

Warning policy

/W4 applies strict diagnostics to the plugin source. /external:anglebrackets /external:W0 suppresses warnings in headers included with angle brackets, including SDK headers. This is broader than directory-only isolation: any project-owned header included with angle brackets may also be treated as external. Prefer quoted includes for project-owned headers.

Do not assume that adding the same SDK directories to both ordinary and external include directory properties reliably establishes directory-specific classification. If precise SDK-only warning suppression is later needed, configure and inspect the effective MSVC command line, avoiding duplicate /I and /external:I entries.

5. IntelliSense troubleshooting

The Qt-enabled plugin compiled, but Visual Studio initially displayed E1696 errors for Qt headers and follow-on undefined identifiers. Rescanning the solution resolved the problem.

- Save OBSPaths.props and select the correct Release | x64 configuration.
- Use Project > Rescan Solution. This was the successful remedy.
- If needed, unload/reload the project to refresh MSBuild imports.
- Verify Project Properties > C/C++ > General > Additional Include Directories.

• Only if errors persist, close Visual Studio and clear the solution-local .vs cache, then reopen.

A successful build and stale IntelliSense errors can coexist. Treat the compiler output and IntelliSense diagnostics as separate evidence when troubleshooting.

6. Dock architecture and lifecycle

The proposed interface uses a QWidget-based dock registered through the public OBS frontend API obs_frontend_add_dock_by_id. It should not patch or embed controls inside OBS private Controls dock implementation.

UI composition

• A compact dock title and current-orientation status label.

• One action button that offers the opposite orientation (Landscape or Portrait).

• Optional timer or frontend-event refresh to reflect changes made elsewhere in OBS.

• Clear disabled state when OBS outputs are active, with a short explanatory message.

Ownership and threading

Create and manipulate QWidget instances only on the Qt GUI thread. Register the dock only after the OBS frontend and Qt application are ready. Avoid blocking the GUI thread. Do not destroy a dock widget while OBS still owns or references it; follow the dock API ownership contract for the exact OBS headers being compiled against.

Use Qt signal/slot connections with lambdas where practical; this can avoid adding a Q_OBJECT macro and a separate MOC build step. Ensure captured objects remain alive for the duration of each connection.

Plugin lifecycle checklist

• obs_module_load: initialize plugin state and register frontend integration at an appropriate point.

• Frontend ready: create/register the dock and connect button events.

• Button click: validate output state, then request the orientation change through the intended supported mechanism.

• Frontend/profile change: refresh label and button text from actual OBS settings.

• obs_module_unload: disconnect callbacks, remove the dock when supported, and release owned resources safely.

7. Orientation switching requirements

Target landscape resolution is 1920 x 1080; target portrait resolution is 1080 x 1920. Both base (canvas) and output (scaled) dimensions must remain internally consistent. Do not modify settings while recording, streaming, replay buffer, or virtual camera outputs are active unless the specific OBS operation is documented as safe.

Important implementation boundary: changing a profile configuration file does not necessarily reconfigure an already-running OBS video subsystem. The plugin must use an appropriate frontend-supported update/reset mechanism, and runtime behavior must be validated against OBS 32.2.1. Avoid presenting raw config writes alone as a guaranteed live resolution switch.

Preserve unrelated profile settings. Handle missing or invalid settings defensively, and avoid overwriting user-customized resolutions without a clear action or confirmation.

8. Build, deploy, and test

Build

```
Configuration: Release | x64
Build > Rebuild Solution
Expected artifact:
E:\VSCODE\obs-orientation-switcher\build\x64\Release\
  obs-orientation-switcher.dll
```

Deployment

Close OBS before replacing a loaded plugin DLL. Copy the newly built DLL to the correct OBS plugin runtime location (in this setup: C:\Program Files\obs-studio\obs-plugins\64bit). Keep a backup of the previously working DLL. Do not copy development headers or import libraries into the OBS runtime folder.

Runtime acceptance checks

- OBS launches without plugin loading errors; inspect OBS log for plugin registration.
- Orientation Switcher dock is available and can be shown, hidden, and docked.
- Button correctly reports current profile orientation on initial load.
- Landscape -> portrait updates both base and output resolution as intended.
- Portrait -> landscape restores both target dimensions.
- Changes persist or revert according to the documented design after OBS restart/profile change.
- Button is disabled or safely refuses changes while an output is active.
- Closing/reopening the dock and exiting OBS cause no crash or use-after-free.

9. Troubleshooting reference

Symptom	Likely cause / next check
C1189 in Qt header	C++17 and /Zc:__cplusplus missing or overridden.
C1083 Qt header missing	Check qt\include and QtCore/Gui/Widgets include paths.
E1696 after successful build	Project > Rescan Solution; verify active config.
LNK1104 Qt6Widgets.lib	Check Qt SDK lib directory and architecture.
DLL fails to load in OBS	Check dependent DLLs, ABI, OBS logs, and x64 build.
Dock missing	Check registration timing, frontend API and log output.
Resolution label stale	Refresh on profile/frontend changes, not only button clicks.
Settings change but video unchanged	Verify supported runtime reconfiguration path.

10. Development decisions and next steps

This setup intentionally retains a small, native Visual Studio project rather than rebuilding OBS Studio. The Qt development SDK is stored locally alongside the minimal OBS SDK. This is a practical strategy for a focused Windows x64 OBS plugin, provided version compatibility and deployment are checked.

- Keep the working property sheet under version control.
- Document the Qt SDK package/version and license obligations for redistribution.
- Compile and run the dock-enabled plugin in OBS 32.2.1.
- Confirm the exact frontend dock API signatures from the local OBS headers.
- Add structured plugin logging and output-state checks.
- Validate video reconfiguration, persistence, and profile switching before release.

Scope note: This document records the environment and observed results from the development session. It is not a certification that the Qt dock implementation has already passed runtime tests.